Zsanleigh Peter F. Serra - BSIT3/C - AppDev - Apr 17, 2026

Role -

I was responsible for building Client 2, the 2nd client application that connects to and interacts with our group's PHP REST API (service.php). While another member handled the Python CLI client, my task was to develop a UI-based client, which I first built as a Node.js CLI application and later remade into just a HTML + JavaScript browser-based UI to serve as a more accessible mock client for demonstration

Task -

1. Node.js CLI Client (client1.js)

I built the initial client as a terminal application using Node.js. It presented a numbered menu to the user and accepted input through the command line using the readline module. The client supported all five core API operations:

Register a new user

Login (with basic session tracking)

Create a seller shop

Read seller information

Update a shop name

2. HTML + JavaScript UI Client (client2.html)

After the CLI was complete, I remade the client into a simple browser-based UI. Instead of a numbered menu, the user selects an action from a dropdown, which shows only the relevant form, keeping the interface slim and focused. All requests are sent using the browser's native fetch API.

Code -

Request Wrapper (sendRequest)

Rather than writing duplicate fetch logic in every function, I did a shared sendRequest(method, url, data) helper. It handles attaching the correct HTTP method, setting Content-Type: application/json, and conditionally adding a request body (skipped for GET requests where params go in the URL instead).

A design I made also here was the error handling inside the function, since the PHP API sometimes returns plain text or HTML on errors instead of JSON. To handle this, I always read the response as text first, then attempt JSON.parse(). If that fails, I return a structured object with the raw text, so the UI can still display something useful instead of crashing.

Session State (globalUserId)

After a successful login, I store the returned user ID in a globalUserId variable. This allows the

seller-related forms to auto-fill the user ID without requiring the user to type it again every time. If they enter an ID manually, that takes priority over the stored one.

Section Toggling (showSection)

The HTML client uses a showSection(name) function tied to the dropdown's onchange event. It iterates through all section IDs and sets display: none on all except the selected one. The output area is also cleared on each switch to avoid showing stale responses from a previous action.

Error Handling

Each action function (doRegister, doLogin, etc.) wraps its request in a try/catch. If the fetch itself throws, like when XAMPP is not running, the catch block calls showError() which displays the error message in the output area instead of silently failing or throwing an unhandled exception.

Challenges -

During basic flow testing, some server-side validation errors returned plain text instead of a proper JSON response. This caused JSON.parse() to throw and break the client. I resolved this by parsing the raw response text first and wrapping the parse attempt in its own try/catch.

the API expects the userId as a query string parameter (?userId=...) rather than a JSON body. This is different from the other POST/PUT operations. I had to handle this as a special case by building the URL string manually and using encodeURIComponent() to ensure the value is URL-safe.

The first version of the HTML client had no try/catch around the fetch calls in each action function. If the server was unreachable, the app would throw an uncaught promise rejection. I caught this and wrapped every action in proper error handling.

Since the requirement was a simple mock UI, I deliberately avoided CSS and kept everything in plain HTML. The main challenge was making the interface usable without it looking too cluttered. I solved this by hid ing all sections by default and only revealing the one matching the dropdown selection.

Contribution -

I provided comments to sections that were not immediately readable, like maybe the fallback to globalUserId and the URL encoding for the GET request, to help other group members and myself understand the logic during code review.